Progress Report: Data Pipeline and First Baseline

Code: https://github.com/S1norin/inno-detecting-license-plates

1 Data pipeline

1.1 Source data

We work on a CCPD2019 subset: 5,000 unique source images. They are tracked with Git LFS in `data/source_data/images/`. The selection records, checksums, and a replacement history are stored next to the images. Eight duplicate photographs from the initial selection were replaced with unique images from the same size and angle groups.

1.2 Splits

- 5,000 originals are split into 3,500 train / 700 val / 800 test.
- The random seed is 42.
- The split is assigned **before** resizing. All three scaled copies of one photo always stay in the same split, so scaled duplicates cannot leak between splits.
- This is a custom split, not the official CCPD benchmark split.

1.3 Scale variants

Each original is saved at three scales: 1.0, 0.75, and 0.5. That gives 15,000 image variants in total. The primary experiments use only the original scale. The full three-scale set is reserved for explicit scale experiments.

1.4 Outputs

The preparation notebook (`data/Data_Preprocessing.ipynb`) writes `data/dataset/` with:

- `images/` and `labels/` — full photos and normalized YOLO boxes, one class (`license_plate`), ready for training.
- `crops_gt/` — plate crops cut from ground-truth boxes, used to evaluate OCR without detector errors.
- `ocr_{train,val,test}.csv` — crop path, target text, scale, and size group.
- `metadata.csv` — 15,000 rows linking every variant to its source, split, scale, box, tilt, and plate text.

- `dataset_original.yaml` and `dataset.yaml` — YOLO config files.
- `run_config.json` — exact parameters and environment of the build.
- `plots/` — distribution and example figures.

Plate height buckets are derived from the training split only: small ≤ 83 px, medium ≤ 112 px, large above that.

1.5 OCR preprocessing

`data/preprocessing.py` exposes three independent crop modes, used as ablations:

- `none` — the crop unchanged.
- `gray_contrast` — grayscale plus CLAHE contrast enhancement.
- `canny` — edge detection with thresholds 100/200.

The function returns BGR or grayscale by default, and three-channel RGB on request.

1.6 Path handling

The committed YOLO configs and image lists contain absolute paths from the machine that built the dataset. After cloning or moving the repo, run `python data/configure_paths.py` to rewrite them for the current machine. This step is easy to forget and is documented in `data/README.md`.

2 Baseline model

2.1 Detection

We start from a pretrained YOLO11s plate detector (`morsetechlab/yolov11-license-plate-detection`, `license-plate-finetune-v1s.pt`). The download is pinned by URL, filename, and SHA-256 in `baseline/common/models.json`, and the local file is verified before every run. No fine-tuning was done in this baseline; the weights are used as-is.

Inference setup (fixed before test evaluation, CPU only):

Setting	Value
Image size	640
Precision / threads	FP32, 1 thread
Confidence floor	0.001 (for AP)
Matching threshold	IoU ≥ 0.5
P/R/F1 threshold	confidence ≥ 0.25
NMS IoU	0.7
Max boxes	300
Seed / batch	42 / 1

Table 1: Detection inference setup.

For a future fine-tuning experiment, `baseline/detection/train_config.yaml` already lists starting hyperparameters: 100 epochs, batch 16, image size 640, seed 42, deterministic mode. A

`train_ready` check verifies the cached weights, config, and all 3,500 training images without starting training.

2.2 Detection results (original scale)

Split	Images	mAP@0.5	Precision	Recall	F1
Validation	700	0.9245	0.9402	0.8986	0.9189
Test	800	0.9073	0.9408	0.8938	0.9167

Table 2: Detection results on the original scale.

By plate size on the test split (mAP@0.5):

Size	Images	mAP@0.5	F1
Small	261	0.8453	0.8969
Medium	277	0.9505	0.9423
Large	262	0.9371	0.9098

Table 3: Test-split detection results by plate size.

The detector is strong overall. Small plates are the clear weak spot.

Reproducibility checks passed: an independent Ultralytics `val()` run agreed with our custom metrics within 0.001, and a full repeat run produced identical prediction records and metrics.

2.3 Recognition

The recognition baseline uses EasyOCR 1.7.2 (Chinese simplified + English, CPU, greedy decoding) through a small pinned adapter. Two protocols are measured:

- **OCR-only** — reads the ground-truth crops. This isolates the recognizer from the detector.
- **End-to-end** — reads the highest-score box from the YOLO detector.

Preprocessing mode and confidence threshold are selected on validation only, by sweeping the threshold from 0.50 to 0.99. Both protocols selected `none` with threshold 0.50, and test was then run once with that fixed policy.

Protocol	Split	Character accuracy	Exact plates
OCR-only	val	0.3580	3 / 700
OCR-only	test	0.3474	0 / 800
End-to-end	val	0.3279	1 / 700
End-to-end	test	0.3240	0 / 800

Table 4: Recognition results with the validation-selected policy.

Preprocessing ablation on validation (character accuracy):

None of the three preprocessing modes helps. Canny is particularly bad because low-contrast crops can produce almost empty edge maps.

Mode	Character accuracy	Coverage
none	0.3580	0.7386
gray_contrast	0.2693	0.5714
canny	0.0328	0.1257

Table 5: Preprocessing ablation on validation.

The detection baseline works well. The recognition baseline is weak, and that is the main current limitation of the system.

3 Problems and observations

3.1 Low OCR scores

The near-zero exact-match scores triggered a validation-only audit (no test data touched). On 100 validation samples we compared the stored adapter against native EasyOCR APIs, alphabet restrictions, and rectification using the annotated plate corners. Nothing fixed whole-plate recognition (character accuracy stayed around 0.35–0.43). The evidence points to poor transfer of this general-purpose OCR checkpoint to plate crops, not to a bug in our pipeline. The next planned experiment is a plate-specific recognizer.

3.2 Limitations

- The split is custom; results are not comparable to official CCPD benchmarks.
- Tilt bins use the CCPD filename proxy and have very unequal sizes.
- The timing probe in the evaluation report covers 100 sampled images, not the full splits.

References

- Dataset format and usage: `data/README.md`
- Detection baseline details and reproduction: `baseline/detection/README.md`
- Recognition baseline details: `baseline/ocr/README.md`
- Combined evaluation (detection + OCR + end-to-end + timing): `baseline/evaluation/REPORT.md`
- Repository: https://github.com/S1norin/inno-detecting-license-plates